

Certified Kubernetes Administrator (CKA) Exam Study Guide

Exam Overview

Exam Details:

- **Duration:** 2 hours
- **Format:** Online, proctored, performance-based test
- **Kubernetes Version:** v1.34 (aligned within 4-8 weeks of K8s release)
- **Cost:** \$445 USD
- **Attempts:** Two exam attempts (includes one retake)
- **Certification Validity:** 2 years
- **Exam Access:** 12 months to schedule and take the exam

Exam Domains and Weightage

The CKA exam is divided into five domains with the following weightage:

Domain	Weightage	Priority
Troubleshooting	30%	Highest
Cluster Architecture, Installation & Configuration	25%	High
Services & Networking	20%	Medium
Workloads & Scheduling	15%	Medium
Storage	10%	Lower

Domain 1: Cluster Architecture, Installation & Configuration (25%)

Kubernetes Components

Control Plane Components

- **kube-apiserver:** Core component that exposes the Kubernetes HTTP API
- **etcd:** Consistent and highly-available key-value store for all API server data
- **kube-scheduler:** Assigns Pods to suitable nodes
- **kube-controller-manager:** Runs controllers to implement Kubernetes API behavior

- **cloud-controller-manager**: Integrates with cloud providers (optional)

Node Components

- **kubelet**: Ensures Pods and their containers are running
- **kube-proxy**: Maintains network rules for Services
- **Container Runtime**: Software responsible for running containers

Addons

- **DNS**: Cluster-wide DNS resolution
- **Web UI (Dashboard)**: Web-based cluster management
- **Container Resource Monitoring**: Collecting and storing metrics
- **Cluster-level Logging**: Centralized container log storage

Role-Based Access Control (RBAC)

Core RBAC Objects

- **Role**: Namespaced permissions for specific namespace resources
- **ClusterRole**: Cluster-wide or namespaced permissions for all namespaces
- **RoleBinding**: Grants Role permissions to users/groups in a namespace
- **ClusterRoleBinding**: Grants ClusterRole permissions cluster-wide

Key RBAC Concepts

```
# Example Role
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
# Example RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
```

name: pod-reader
apiGroup: rbac.authorization.k8s.io

Default Roles and Bindings

- **cluster-admin**: Super-user access to all resources
- **admin**: Admin access within a namespace
- **edit**: Read/write access to most objects in namespace
- **view**: Read-only access to most objects in namespace

kubeadm Cluster Management

Cluster Initialization

- **kubeadm init**: Initialize control plane node
- **kubeadm init phase**: Run specific initialization phases
- **kubeadm join**: Join nodes to cluster
- **kubeadm join phase**: Run specific join phases

Key kubeadm Commands

Initialize cluster

```
kubeadm init --pod-network-cidr=192.168.0.0/16
```

Join worker node

```
kubeadm join <control-plane-ip>:<port> --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Generate join token

```
kubeadm token create --print-join-command
```

Upgrade cluster

```
kubeadm upgrade plan
```

```
kubeadm upgrade apply v1.34.0
```

Certificate management

```
kubeadm certs renew all
```

Reset cluster

```
kubeadm reset
```

Certificate Management

- Certificates stored in /etc/kubernetes/pki/
- Default validity: 1 year for CA, 1 year for other certificates
- Use kubeadm certs renew to renew certificates
- CA certificates: 10 years validity

High Availability Configuration

HA Control Plane Options

- **Stacked etcd topology:** etcd on control plane nodes
- **External etcd topology:** etcd on separate nodes
- **Load balancer:** Front multiple control plane nodes

HA Requirements

- Minimum 3 control plane nodes for quorum
- Load balancer for API server access
- Consistent configuration across control plane nodes

Helm Package Manager

Helm Concepts

- **Chart:** Kubernetes application package
- **Release:** Chart instance running in cluster
- **Repository:** Chart storage location
- **Values:** Configuration parameters

Essential Helm Commands

Install chart

```
helm install <release-name> <chart>
```

List releases

```
helm list
```

Upgrade release

```
helm upgrade <release-name> <chart>
```

Rollback release

```
helm rollback <release-name> <revision>
```

Add repository

```
helm repo add <name> <url>
```

Search charts

```
helm search repo <keyword>
```

Extension Interfaces

Container Runtime Interface (CRI)

- Interface between kubelet and container runtime

- Supported runtimes: containerd, CRI-O, Docker (deprecated)

Container Network Interface (CNI)

- Network plugin interface
- Popular CNI plugins: Calico, Flannel, Weave, Cilium

Container Storage Interface (CSI)

- Storage plugin interface
- Enables third-party storage drivers
- Replaces in-tree volume plugins

Custom Resource Definitions (CRDs)

CRD Management

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.stable.example.com
spec:
  group: stable.example.com
  versions:
    - name: v1
      served: true
      storage: true
  scope: Namespaced
  names:
    plural: crontabs
    singular: crontab
    kind: CronTab
    shortNames:
      - ct
```

Operator Pattern

- Extends Kubernetes API with custom controllers
- Automates management of complex stateful applications
- Uses CRDs to define new resource types

Domain 2: Troubleshooting (30%)

Cluster Troubleshooting

Node Issues

- **Node Not Ready:** Check node status and conditions
- **Node Pressure:** Memory, disk, or PID pressure
- **Kubelet Issues:** Container runtime connectivity problems

Control Plane Issues

- **API Server:** Authentication, authorization, admission control
- **Scheduler:** Pod scheduling failures
- **Controller Manager:** Reconciliation loop issues
- **etcd:** Database connectivity and performance

Common Troubleshooting Commands

Get cluster info

```
kubectl cluster-info
```

Check node status

```
kubectl get nodes -o wide
```

```
kubectl describe node <node-name>
```

Check system pods

```
kubectl get pods -n kube-system
```

Check events

```
kubectl get events --sort-by='.lastTimestamp'
```

Check logs

```
kubectl logs <pod-name> -n <namespace>
```

```
kubectl logs <pod-name> -n <namespace> --previous
```

Pod Troubleshooting

Pod States

- **Pending:** Waiting for scheduling or resource allocation
- **Running:** Container(s) running successfully
- **Succeeded:** All containers terminated successfully
- **Failed:** At least one container failed
- **Unknown:** Pod status cannot be obtained

Common Pod Issues

- **ImagePullBackOff:** Image download failed
- **CrashLoopBackOff:** Container repeatedly failing
- **OOMKilled:** Out of memory kill

- **Evicted:** Node pressure conditions

Pod Debugging

Describe pod

kubectl describe pod <pod-name>

Get pod logs

kubectl logs <pod-name>

kubectl logs <pod-name> --previous

Execute commands in pod

kubectl **exec** -it <pod-name> -- /bin/bash

Port forward for testing

kubectl port-forward <pod-name> <local-port>:<pod-port>

Debug pod with ephemeral container

kubectl debug <pod-name> -it --image=busybox

Container Troubleshooting

Container Lifecycle Issues

- **Container start failures:** Check image, entrypoint, volumes
- **Resource limits:** CPU/memory constraints
- **Health checks:** Liveness/readiness probe failures

Container Debugging

Get container logs

kubectl logs <pod-name> -c <container-name>

Execute in specific container

kubectl **exec** -it <pod-name> -c <container-name> -- /bin/bash

Check container status

kubectl get pods <pod-name> -o **jsonpath**='{.status.containerStatuses[*]}'

Service and Networking Troubleshooting

Service Issues

- **Service not reachable:** Check endpoint creation
- **DNS resolution:** Verify CoreDNS pods and configuration
- **Network policies:** Verify NetworkPolicy rules

DNS Troubleshooting

```
# Check CoreDNS
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

```
# Test DNS resolution
kubectl exec -it <pod-name> -- nslookup kubernetes.default
```

```
# Check DNS config
kubectl get configmap coredns -n kube-system -o yaml
```

Monitoring and Logging

Resource Monitoring

- Metrics Server: For resource metrics (CPU, memory)
- `kubectl top`: Node and pod resource usage
- Resource metrics pipeline: Prometheus integration

Logging Architecture

- Container logs: stdout/stderr streams
- Node-level logging: Systemd journal
- Cluster-level logging: Centralized log aggregation

Logging Commands

```
# Check resource usage
kubectl top nodes
kubectl top pods -n <namespace>
```

```
# View logs
kubectl logs -f <pod-name>
kubectl logs <pod-name> --tail=100
```

```
# Follow logs across pods
kubectl logs -l app=<app-name> --tail=10
```

Cluster Component Debugging

etcd Troubleshooting

```
# Check etcd health
etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key
endpoint health
```

```
# View etcd data
etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key get / --
prefix
```


API Server Troubleshooting

```
# Check API server status
curl -k https://<apiserver-ip>:6443/healthz
curl -k https://<apiserver-ip>:6443/readyz
```

```
# Check API server logs
journalctl -u kube-apiserver
```

Domain 3: Services & Networking (20%)

Kubernetes Services

Service Types

- **ClusterIP**: Accessible within cluster (default)
- **NodePort**: Exposes service on each node's IP at static port
- **LoadBalancer**: Integrates with cloud provider load balancers
- **ExternalName**: Maps service to external DNS name

Service Configuration

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
      nodePort: 30007 # Only for NodePort type
```

Service Discovery

- **DNS**: CoreDNS provides service name resolution
- **Environment variables**: Kubernetes sets service environment variables
- **Endpoints**: Service backend IP addresses and ports

CoreDNS

CoreDNS Configuration

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health {
        lameduck 5s
      }
      ready
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
      }
      prometheus :9153
      forward . /etc/resolv.conf
      cache 30
      loop
      reload
      loadbalance
    }
```

CoreDNS Troubleshooting

- Check CoreDNS deployment in kube-system namespace
- Verify DNS resolution from within pods
- Check CoreDNS logs for errors
- Ensure upstream DNS servers are accessible

Network Policies

Network Policy Basics

- Control traffic flow at IP/port level (Layer 3/4)
- Application-centric traffic control
- Implemented by network plugin (CNI)

Network Policy Structure

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
```

```
podSelector:
  matchLabels:
    role: db
policyTypes:
- Ingress
- Egress
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        project: myproject
  - podSelector:
      matchLabels:
        role: frontend
ports:
- protocol: TCP
  port: 6379
egress:
- to:
  - ipBlock:
      cidr: 10.0.0.0/24
ports:
- protocol: TCP
  port: 5978
```

Network Policy Types

- **Default Deny Ingress:** policyTypes: [Ingress] with no ingress rules
- **Default Deny Egress:** policyTypes: [Egress] with no egress rules
- **Default Deny All:** Both Ingress and Egress with no rules
- **Allow All:** Empty selectors with allow rules

Ingress

Ingress Resources

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: foo.bar.com
    http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
```

port:
number: 80

Ingress Controllers

- **NGINX Ingress Controller:** Most popular
- **Traefik:** Cloud-native reverse proxy
- **Istio Gateway:** Service mesh ingress
- **AWS ALB Ingress:** AWS Application Load Balancer

Gateway API

Gateway Classes

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: GatewayClass
metadata:
  name: istio
spec:
  controllerName: istio.io/gateway-controller
```

Gateway Resources

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: Gateway
metadata:
  name: external-gateway
spec:
  gatewayClassName: istio
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
          from: All
```

EndpointSlices

EndpointSlice Structure

```
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: example-abc
  labels:
    kubernetes.io/service-name: example
addressType: IPv4
endpoints:
  - addresses:
    - "10.1.2.3"
```

```
conditions:
  ready: true
ports:
- name: http
  protocol: TCP
  port: 80
```

Domain 4: Workloads & Scheduling (15%)

Pods

Pod Specification

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
  resources:
    requests:
      memory: "64Mi"
      cpu: "250m"
    limits:
      memory: "128Mi"
      cpu: "500m"
  livenessProbe:
    httpGet:
      path: /
      port: 80
    initialDelaySeconds: 3
    periodSeconds: 3
  readinessProbe:
    httpGet:
      path: /
      port: 80
    initialDelaySeconds: 3
    periodSeconds: 3
```

Pod Lifecycle

- **Pending:** Pod accepted but containers not ready
- **Running:** At least one container is running

- **Succeeded:** All containers terminated successfully
- **Failed:** All containers terminated, at least one failed
- **Unknown:** Pod state cannot be determined

Container Types

- **Init Containers:** Run during Pod startup, complete before app containers
- **App Containers:** Main application containers
- **Ephemeral Containers:** For debugging running Pods
- **Sidecar Containers:** Auxiliary services (v1.33+ with restartPolicy: Always)

Deployments

Deployment Specification

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Deployment Operations

Create deployment

```
kubectl apply -f deployment.yaml
```

Scale deployment

```
kubectl scale deployment nginx-deployment --replicas=5
```

Update deployment

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1
```

Rollback deployment

```
kubectl rollout undo deployment/nginx-deployment
```

```
# Check rollout status
kubectl rollout status deployment/nginx-deployment

# View rollout history
kubectl rollout history deployment/nginx-deployment
```

StatefulSets

StatefulSet Specification

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
              name: web
          volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

StatefulSet Features

- **Stable network identity:** Pods have stable DNS names
- **Stable storage:** Persistent volumes for each pod
- **Ordered deployment and scaling:** Sequential creation/deletion
- **Ordered rolling updates:** Update pods in reverse order

DaemonSets

DaemonSet Specification

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      containers:
        - name: fluentd-elasticsearch
          image: quay.io/fluentd_elasticsearch/fluentd:v2.5.2
          resources:
            limits:
              memory: 200Mi
            requests:
              cpu: 100m
              memory: 200Mi
          volumeMounts:
            - name: varlog
              mountPath: /var/log
            - name: varlibdockercontainers
              mountPath: /var/lib/docker/containers
              readOnly: true
      terminationGracePeriodSeconds: 30
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
        - name: varlibdockercontainers
          hostPath:
            path: /var/lib/docker/containers
```

Jobs and CronJobs

Job Specification

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
```



```
containers:
- name: pi
  image: perl
  command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
restartPolicy: Never
backoffLimit: 4
```

CronJob Specification

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "*/* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

Pod Scheduling

Node Selection

```
spec:
  nodeSelector:
    kubernetes.io/os: linux
  hardware: gpu
```

Node Affinity

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/os
                operator: In
                values:
                  - linux
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 1
          preference:
```

```
matchExpressions:
- key: hardware
  operator: In
values:
- gpu
```

Pod Affinity and Anti-Affinity

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: security
                operator: In
                values:
                  - S1
          topologyKey: topology.kubernetes.io/zone
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
                - key: app
                  operator: In
                  values:
                    - nginx
            topologyKey: kubernetes.io/hostname
```

Taints and Tolerations

```
# Taint a node
kubectl taint nodes node1 key1=value1:NoSchedule
```

```
# Toleration in pod
spec:
  tolerations:
    - key: "key1"
      operator: "Equal"
      value: "value1"
      effect: "NoSchedule"
```

Resource Management

Resource Requests and Limits

```
spec:
  containers:
    - name: app
      image: app:latest
      resources:
```

```
requests:
  memory: "64Mi"
  cpu: "250m"
limits:
  memory: "128Mi"
  cpu: "500m"
```

Quality of Service Classes

- **Guaranteed:** Requests equal limits for all containers
 - **Burstable:** Requests less than limits
 - **BestEffort:** No requests or limits specified
-

Domain 5: Storage (10%)

Persistent Volumes (PV)

PV Specification

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: slow
  hostPath:
    path: "/tmp/pv1"
```

Access Modes

- **ReadWriteOnce (RWO):** Mounted read-write by single node
- **ReadOnlyMany (ROX):** Mounted read-only by many nodes
- **ReadWriteMany (RWX):** Mounted read-write by many nodes

Reclaim Policies

- **Retain:** Manual reclamation required
- **Delete:** Delete volume when claim deleted
- **Recycle:** Simple volume recycling (deprecated)

Persistent Volume Claims (PVC)

PVC Specification

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc1
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: fast
```

Using PVC in Pod

```
spec:
  containers:
    - name: app
      image: app:latest
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pvc1
```

Storage Classes

Storage Class Specification

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
  zone: us-east-1a
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
```

Volume Binding Modes

- **Immediate:** Create and bind PVC immediately
- **WaitForFirstConsumer:** Delay binding until pod scheduled

Volume Types

emptyDir

```
spec:
  containers:
    - name: app
      image: app:latest
      volumeMounts:
        - name: cache
          mountPath: /cache
  volumes:
    - name: cache
      emptyDir: {}
```

hostPath

```
spec:
  volumes:
    - name: host-path
      hostPath:
        path: "/tmp"
        type: DirectoryOrCreate
```

ConfigMap and Secret

```
spec:
  containers:
    - name: app
      image: app:latest
      volumeMounts:
        - name: config
          mountPath: /config
        - name: secrets
          mountPath: /secrets
  volumes:
    - name: config
      configMap:
        name: app-config
    - name: secrets
      secret:
        secretName: app-secrets
```

Storage CSI Drivers

CSI Volume Configuration

```
spec:
  containers:
    - name: app
      image: app:latest
      volumeMounts:
        - name: csi-volume
          mountPath: /data
```

```
volumes:
- name: csi-volume
  csi:
    driver: csi-driver.example.com
    volumeHandle: volume-123
    readOnly: false
```

Volume Snapshot and Cloning

Volume Snapshot

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: new-snapshot-pvc
spec:
  source:
    persistentVolumeClaimName: pvc1
```

Volume Clone

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cloned-pvc
spec:
  dataSource:
    name: new-snapshot-pvc
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Essential kubectl Commands

Basic Operations

Get resources

```
kubectl get pods
kubectl get nodes
kubectl get namespaces
kubectl get deployments
kubectl get services
```

Describe resources

```
kubectl describe pod <pod-name>
```

```
kubectl describe node <node-name>
kubectl describe deployment <deployment-name>
```

Create resources

```
kubectl apply -f <file>
kubectl create namespace <namespace>
```

Delete resources

```
kubectl delete pod <pod-name>
kubectl delete -f <file>
```

Edit resources

```
kubectl edit deployment <deployment-name>
```

Get logs

```
kubectl logs <pod-name>
kubectl logs -f <pod-name>
kubectl logs <pod-name> --previous
```

Execute commands

```
kubectl exec -it <pod-name> -- /bin/bash
kubectl exec <pod-name> -- ls -la
```

Port forward

```
kubectl port-forward <pod-name> <local-port>:<pod-port>
```

Advanced Operations

Scale resources

```
kubectl scale deployment <deployment-name> --replicas=3
```

Update resources

```
kubectl set image deployment/<deployment-name> <container>=<image:tag>
```

Rollout management

```
kubectl rollout status deployment/<deployment-name>
kubectl rollout history deployment/<deployment-name>
kubectl rollout undo deployment/<deployment-name>
```

Copy files

```
kubectl cp <pod-name>:<path> <local-path>
kubectl cp <local-path> <pod-name>:<path>
```

Drain node

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

Cordon/uncordon node

```
kubectl cordon <node-name>
kubectl uncordon <node-name>
```

Top resources

```
kubectl top nodes
kubectl top pods -n <namespace>
```

Labels and Selectors

Add labels

```
kubectl label pod <pod-name> environment=production
```

Remove labels

```
kubectl label pod <pod-name> environment-
```

Select by labels

```
kubectl get pods -l app=nginx
```

```
kubectl get pods -l 'app in (nginx,apache)'
```

Select by annotations

```
kubectl get pods -l 'app.kubernetes.io/name=nginx'
```

Namespace Operations

Create namespace

```
kubectl create namespace <namespace>
```

Switch context

```
kubectl config set-context --current --namespace=<namespace>
```

Get all resources in namespace

```
kubectl get all -n <namespace>
```

Set resource quota

```
kubectl create quota <quota-name> --hard=cpu=1,memory=1Gi --namespace=<namespace>
```

Exam Tips and Best Practices

Time Management

- 30 minutes on Troubleshooting (30% weightage)
- 25 minutes on Cluster Architecture (25% weightage)
- 20 minutes on Services & Networking (20% weightage)
- 15 minutes on Workloads & Scheduling (15% weightage)
- 10 minutes on Storage (10% weightage)

Common Mistakes to Avoid

1. **Not using the correct context:** Always check which cluster/namespace you're working with
2. **Incorrect YAML syntax:** Validate YAML structure carefully
3. **Missing required fields:** Check API documentation for required fields
4. **Not checking pod status:** Always verify pods are running before proceeding
5. **Overlooking events:** Use `kubectl describe` and `events` to troubleshoot

Quick Reference Commands

Check cluster health

kubectl get nodes && kubectl get pods -n kube-system

Find all pods by label

kubectl get pods -l <label>=<value> --all-namespaces

Check resource usage

kubectl top nodes && kubectl top pods --all-namespaces

View events sorted by time

kubectl get events --sort-by='.lastTimestamp'

Check kubelet status

systemctl status kubelet

Check container runtime

docker ps || crictl ps

Essential Files and Directories

- **Kubeconfig:** ~/.kube/config
 - **Certificate directory:** /etc/kubernetes/pki/
 - **Manifest directory:** /etc/kubernetes/manifests/ (static pods)
 - **Kubelet config:** /var/lib/kubelet/config.yaml
 - **Cluster info:** /etc/kubernetes/admin.conf
-

Practice Scenarios

Scenario 1: Pod Troubleshooting

A pod is in CrashLoopBackOff state. Troubleshoot and fix the issue.

Scenario 2: Service Configuration

Configure a service to expose a deployment with proper endpoints.

Scenario 3: Storage Configuration

Set up persistent storage with appropriate storage class and access modes.

Scenario 4: RBAC Configuration

Create roles and bindings to grant appropriate permissions to users and service accounts.

Scenario 5: Network Policy

Implement network policies to restrict pod-to-pod communication.

Scenario 6: Cluster Upgrade

Upgrade a Kubernetes cluster from one version to another using kubeadm.

Scenario 7: DNS Troubleshooting

Debug DNS resolution issues in the cluster.

Scenario 8: Resource Management

Configure resource limits and requests for applications.

Scenario 9: Pod Scheduling

Use various scheduling features like affinity, taints, and tolerations.

Scenario 10: Backup and Recovery

Backup and restore cluster configuration and data.

Additional Resources

Official Documentation

- [Kubernetes Documentation](#)
- [kubectl Command Reference](#)

- [kubeadm Documentation](#)
- [CKA Curriculum](#)

Practice Platforms

- [Killer.sh CKA Simulator](#)
- [Kubernetes Playground](#)
- [Katacoda Kubernetes Tutorials](#)

Community Resources

- [Kubernetes Slack](#)
- [Kubernetes Forums](#)
- [Stack Overflow - kubernetes tag](#)

This study guide is based on the official CKA curriculum and Kubernetes documentation as of Kubernetes v1.34. Always refer to the official documentation for the most up-to-date information.